WEEKLY MINI PROJECT 04 · TEAM 3

Bytes for Futures Assessment Platform

Work Package · Backend B

Backend | NestJS + Prisma | Judge0

PROJECT	TEAM LEADER	OWNER
Bytes for Futures Assessment Platform	**Ali Mikdad**	**Majed Hmoud**

ROLE	AREA
Backend developer	**Test content · Publishing · Seed content · Code Execution Service**

> *What the instructor manages, and the only part of the backend that touches student code.*

> **Use the class pattern:** the same NestJS layout you built in B4F Board: module, controller, service, DTOs, and `PrismaService` injected into the service (as in the Session 21 cutover). Controllers only pass data; services hold the rules; database access only in services; format checks only in DTOs.

> **Never** run student code on our server: no `eval`, `new Function`, `vm`, `child_process`. Every Run and Submit goes through `CodeExecutionService`, which uses mock mode or the external Judge0 service.

> **Full reference:** this package is a summary of `docs/TASKS.md`. Every name, endpoint, shape and message is fixed in `docs/API.md` and `docs/PROJECT-STRUCTURE.md`. If this package and the docs ever disagree, the docs win; tell Ali.

1 Your Work at a Glance

#	Task	Main file(s)
T1	Execution spike: which host works without a VPN (Phase 0)	`execution/SPIKE.md`
T2	Content DTOs, publish and unpublish	`tests/dto/*`, `tests/instructor-tests.service.ts`
T3	Questions and problems services	`tests/questions.service.ts`, `tests/problems.service.ts`, `tests/mappers/*`
T4	C1 · Types, interface, mock provider	`execution/execution.types.ts`, `providers/mock.provider.ts`
T5	C2 · Harness builder and result parser	`execution/harness/*`
T6	C3 · Output comparer	`execution/output-comparer.ts`
T7	C4 · Error normalizer	`execution/error-normalizer.ts`, `__fixtures__/`
T8	C5 · Judge0 provider	`execution/providers/judge0.provider.ts`

#	Task	Main file(s)
T9	C6 · `run()` and `submit()`	`execution/code-execution.service.ts`
T10	Coding problems seed	`prisma/seed-data/coding-problems.ts`
T11	MCQ seed and shuffled order	`prisma/seed-data/mcq-questions.ts`, `tests/attempt-order.util.ts`

2 Work Without Waiting

PHASE 0 · DAY 1 MORNING

Stubs everywhere

Ali and Esmaeel push both skeletons. Every endpoint answers with the example JSON from `API.md`; every route shows a placeholder.

PHASE 1 · DAYS 1–2

Build alone

Everyone works in their own files, against the fixed names. Replace stubs only inside your own module.

PHASE 2 · END OF DAY 2

Real integration

Every stub replaced, every context on the real endpoints, every message checked against `API.md`.

Your work depends on `docs/API.md` and `docs/PROJECT-STRUCTURE.md`, not on someone's unfinished code. If a dependency is not merged yet, code against the fixed name and test with the stub.

3 Your Tasks, Step by Step

T1 Execution spike (Phase 0, ~2 h, in parallel)

`api/src/execution/SPIKE.md`

Goal: know **which execution host works from Syria without a VPN**, before anyone writes the Judge0 provider.

1. Turn off any VPN.
2. For each candidate, in this order, record: reachable? needs an account/key? can the account be created from Syria? `GET /languages` works? newest Node.js language id? does `wait=true` work? one test submission works?
3. Write the results in `api/src/execution/SPIKE.md` and post the chosen `JUDGE0_URL`, auth header and language id in the team chat (**not** the key).
4. Collect one real `stderr` / result sample for each error case into `__fixtures__/` (used in T7).
5. If nothing works without a VPN: we stay in mock mode, and Ali tells the instructor.

#	Candidate	Notes
1	A public Judge0 CE instance (for example the one linked from the official Judge0 site)	No key if open. Check rate limits.
2	Judge0 CE on RapidAPI (free Basic plan)	Needs a RapidAPI account and key (`X-RapidAPI-Key`, `X-RapidAPI-Host`). Check that sign-up works from Syria.
3	Another Judge0 host the instructor approves	—
4	Piston public API	Different API shape: only if 1–3 all fail. A `piston.provider.ts` would implement the same provider interface.

> Pick the newest Node.js language from `GET /languages`. Old Node versions (for example 12) do not support `?.` and `??`.

✔ Done when: the spike result is written in `SPIKE.md` and posted.

T2 Content DTOs, publish and unpublish

`tests/dto/question.dto.ts` · `tests/dto/problem.dto.ts` · `tests/dto/test-case.dto.ts` · `tests/instructor-tests.service.ts`

1. DTOs with nested validation (`@ValidateNested`, `@Type`, `@ArrayMinSize`, `@ArrayMaxSize`). Messages from `API.md`, section 7 (table in T3).
2. Rules that need more than one field (exactly one correct option, at least 1 sample and 3 hidden) are checked in the service, before saving.
3. `findAll()` with counts and `isContentLocked` (`attemptStateService.hasAttempts`).
4. `publish()`: R-3 → `409` `An MCQ test needs at least 5 questions before it can be published.` / `A coding test needs at least 1 problem before it can be published.` Already published → return unchanged.
5. `unpublish()`: `expireOverdue()` first, then R-5 → `409` `This test cannot be unpublished while a student is taking it.`
6. `assertContentEditable(testId)`: R-4 (`409`). Called by add, replace, and delete.
7. Log: `test published`, `test unpublished`, `content change rejected`.

✔ Done when: publish and unpublish follow R-3 and R-5; content is locked after the first attempt (R-4).

T3 Questions and problems services

`tests/questions.service.ts` · `tests/problems.service.ts` · `tests/mappers/instructor-test.mapper.ts`

1. `QuestionsService` / `ProblemsService`: create, replace (delete old options or test cases and create new ones **in one transaction**), remove. Wrong test type → `400`.
2. `instructor-test.mapper.ts`: instructor shapes from `API.md` 5.3–5.6.

Problem	Status	Message
Unknown test / question / problem	404	`Test not found.` / `Question not found.` / `Problem not found.`
Question added to a coding test	400	`Questions can only be added to an MCQ test.`
Problem added to an MCQ test	400	`Problems can only be added to a coding test.`
Question text missing	400	`Question text is required.`
Fewer than 2 or more than 6 options	400	`A question needs between 2 and 6 options.`
An option text is empty	400	`Every option needs text.`
Not exactly one correct option	400	`A question needs exactly one correct option.`
Title or description missing	400	`Problem title is required.` / `Problem description is required.`
Function name not a valid JavaScript name	400	`Function name must be a valid JavaScript name, for example twoSum.`
Points not a whole number from 1 to 100	400	`Points must be a whole number from 1 to 100.`
`args` is not an array	400	`Test case arguments must be a JSON array.`
No sample case	400	`A problem needs at least 1 sample test case.`
Fewer than 3 hidden cases	400	`A problem needs at least 3 hidden test cases.`
Any student has an attempt for this test	409	`Questions and problems cannot be changed after a student has started this test.`

✔ Done when: questions and problems are added, replaced, and deleted with every rule and message from `API.md`.

T4 C1 · Types, interface, mock provider

`execution/execution.types.ts` · `providers/execution-provider.interface.ts` · `providers/mock.provider.ts`

Build the execution service in this order (T4 → T9). Each step is one small PR.

1. Types and the provider interface (section 5 of this package).
2. `MockExecutionProvider` follows R-23 exactly: Run → `status: "MOCK"`, message `[MOCK] Code received: 14 lines. Real execution is not enabled.` (the real line count), `actualOutput: null`; empty code → error. Submit → every hidden test passes when the code contains `function <functionName>(` (spaces allowed before `(`), otherwise every hidden test fails.
3. `CodeExecutionService.getMode()` reads `EXECUTION_MODE` once at startup. `mode` is in every result.

✔ Done when: mock mode follows R-23 and every result carries `mode`.

T5 C2 · Harness builder and result parser

`execution/harness/harness.builder.ts` · `harness-result.parser.ts`

1. `harness.builder.ts`: **student code first** (so error lines are the student's real lines), then the harness. `functionName` only after the identifier check (`^[A-Za-z_$][A-Za-z0-9_$]*$`); cases with `JSON.stringify`. The harness gets **only the arguments**, never expected outputs.
2. `harness-result.parser.ts`: finds the **last** `__B4F_RESULT__` line and parses it.
3. A return value that cannot become JSON is an error: `Your function returned a value that cannot be compared (for example a function or a circular object).`

```
The harness (simplified; twoSum stands for the problem's functionName)
// ===== student code above (starts at line 1) =====
;(function __b4fHarness() {
  const __cases = /* JSON array of args arrays */;
  const __logs = [];
  const __origLog = console.log;
  const __origError = console.error;
  console.log = (...a) => __logs.push(a.map(String).join(' '));
  console.error = console.log;
  const __out = { found: typeof twoSum === 'function', results: [] };
  if (__out.found) {
    for (const args of __cases) {
      try {
        const value = twoSum(...args);
        __out.results.push({ ok: true, value: value === undefined ? { __undefined: true } : value });
      } catch (e) {
        __out.results.push({ ok: false, error: { type: e && e.name, message: e && e.message, stack: e &&
e.stack } });
      }
    }
  }
  __out.logs = __logs.slice(0, 50);
  console.log = __origLog;
  console.error = __origError;
  __origLog('__B4F_RESULT__' + JSON.stringify(__out));
})();
```

✔ Done when: the built file runs the student function on every case and prints one result line.

T6 C3 · Output comparer

`execution/output-comparer.ts`

1. Arrays: same length, same order, every item equal.
2. Objects: same keys (key order does not matter), every value equal.
3. Numbers: equal when `Math.abs(a - b) < 1e-9` (so `0.1 + 0.2` equals `0.3`).
4. Strings, booleans, `null`: strict equality. `undefined` (`{ __undefined: true }`) is equal only to an expected value that is also missing.
5. Unit-test it with: equal arrays, different order, nested objects with different key order, `0.1 + 0.2` vs `0.3`, `"1"` vs `1`, `null` vs missing, empty array vs empty object.

✔ Done when: output comparison passes all the unit tests above.

T7 C4 · Error normalizer

`execution/error-normalizer.ts` · `execution/__fixtures__/`

The student sees **the error type, a short message, and their own line number**. Never a raw stack trace, never `/box/script.js`, never our harness.

1. A line number larger than the number of lines in the student code is dropped (`line: null`).
2. File paths and `node:internal` frames are removed from every message. `message` is cut to 300 characters.
3. Unit-test the fixtures (syntax error, `TypeError`, `ReferenceError`, thrown string, missing function, infinite loop): right `status`, `error.type`, `error.message`, and the **student's** line number.

Source	How it is read
Error thrown inside the student function	From the harness: `error.type`, `error.message`, and the first `:<line>:<col>` in `error.stack`
Syntax error (the harness never ran)	No `__B4F_RESULT__` line. Read `stderr`: the first `:<line>` after the file name, and the line matching `^(\w*Error): (.+)$`
Judge0 status 6	Same as syntax error, from `compile_output`
Judge0 status 7–12 with a result line	Use the result line
Judge0 status 7–12 without a result line	Read `stderr`; `SyntaxError` → `SYNTAX_ERROR`, otherwise `RUNTIME_ERROR`
Judge0 status 13, 14, HTTP error, timeout	`503`

`status`	`message`
`PASSED`	`Your function returned the expected output.`
`WRONG_ANSWER`	`Your function returned a different value than expected.`
`SYNTAX_ERROR`	`Your code has a syntax error.`
`RUNTIME_ERROR`	`Your code threw an error while running.`
`FUNCTION_NOT_FOUND`	`No function named <functionName> was found. Declare it like this: function <functionName>(...) { }`
`TIME_LIMIT_EXCEEDED`	`Your code took too long. Check for an infinite loop.`
`MOCK`	`[MOCK] Code received: <n> lines. Real execution is not enabled.`

✔ Done when: every error type gives a readable message with the student's line number.

T8 C5 · Judge0 provider

`execution/providers/judge0.provider.ts`

1. `POST /submissions?base64_encoded=true` with `language_id`, `source_code`, `cpu_time_limit: 2`, `wall_time_limit: 5`. Then `wait=true` if the spike showed it works, otherwise poll `GET /submissions/:token?base64_encoded=true` every 500 ms until the status is no longer 1 or 2.
2. Auth header and token only from `.env` (`JUDGE0_URL`, `JUDGE0_AUTH_HEADER`, `JUDGE0_AUTH_TOKEN`, `JUDGE0_LANGUAGE_ID`), plus the optional second header `JUDGE0_EXTRA_HEADER` / `JUDGE0_EXTRA_VALUE` (hosted services such as RapidAPI need two). Timeout `EXECUTION_TIMEOUT_MS` with `AbortController`.
3. Status 13/14, HTTP error, or timeout → `ServiceUnavailableException` (503): `The code execution service is not reachable right now. Your run was not counted. Please try again.` (Run) / `...Your submission was not saved. Please try again.` (Submit).
4. Use Node's built-in `fetch` (Node 18+). No new package. The browser never talks to Judge0.

✔ Done when: in Judge0 mode a correct solution passes and a wrong one fails; an unreachable host gives `503` with the fixed message.

T9 C6 · `run()` and `submit()`

`execution/code-execution.service.ts`

1. `run(problem, code)`: sample case only → `ExecutionRunOutcome` with input, expected, actual, logs, error.
2. `submit(problem, code)`: hidden cases only → `{ mode, passedCount, totalCount }`. Syntax error, missing function, or timeout → `passedCount: 0`.
3. **Hidden expected values never leave this service** (not to Judge0, not in the outcome).
4. Log provider errors and timeouts (never the code, never hidden data).

✔ Done when: Rami's `CodingService` can call `run` and `submit` and never sees hidden data.

T10 Coding problems seed

`api/prisma/seed-data/coding-problems.ts`

1. 2 problems (for example **Two Sum** and **Reverse Words**), each with 1 sample and at least 4 hidden cases.
2. Run every case through the Judge0 provider once to make sure the expected values are right.

✔ Done when: the two seeded problems pass with a reference solution and fail with an empty function.

T11 MCQ seed and shuffled order

`api/prisma/seed-data/mcq-questions.ts` · `api/src/tests/attempt-order.util.ts`

1. `seed-data/mcq-questions.ts`: at least 10 IQ questions and 10 HTML & CSS questions, each with 4 options and exactly one correct. Check every answer twice.
2. `tests/attempt-order.util.ts` (pure functions, no Prisma):
   - `buildAttemptOrder(questions)` → `{ questionOrder: number[], optionOrder: Record<string, number[]> }`, Fisher–Yates with `crypto.randomInt`.
   - `applyQuestionOrder(questions, order)` and `applyOptionOrder(options, order)`: return a new array in the stored order; items missing from the order go last (safety).
3. Unit-test: every id appears exactly once; two calls give different orders (most of the time); applying an order is stable.

✔ Done when: the MCQ seed has 10+ correct questions per test, and `attempt-order.util.ts` passes its unit tests.

4 Fixed Types You Own

```
api/src/execution/execution.types.ts
interface ExecutionProvider {
  readonly mode: 'MOCK' | 'JUDGE0';
  execute(input: { code: string; functionName: string; cases: unknown[][] }): Promise<ProviderRawResult>;
}

type ExecutionRunOutcome = {
  mode: 'MOCK' | 'JUDGE0';
  status: 'PASSED' | 'WRONG_ANSWER' | 'SYNTAX_ERROR' | 'RUNTIME_ERROR' | 'FUNCTION_NOT_FOUND' |
'TIME_LIMIT_EXCEEDED' | 'MOCK';
  message: string;
  input: unknown[];
  expectedOutput: unknown;
  actualOutput: unknown | null;
  logs: string[];
  error: { type: string; message: string; line: number | null } | null;
  timeMs: number | null;
};

type ExecutionSubmitOutcome = {
  mode: 'MOCK' | 'JUDGE0';
  passedCount: number;
  totalCount: number;
};
```

Used by	Name
Rami	`CodeExecutionService.run(problem, code)`, `submit(problem, code)`, `getMode()`
Rami	`buildAttemptOrder(questions)`, `applyQuestionOrder()`, `applyOptionOrder()`

You use from Rami (stubs from Phase 0): `AttemptStateService.hasAttempts`, `hasActiveAttempts`, `expireOverdue`. From Esmaeel: guards, pipe, filter, Prisma.

5 Rules for You

- Only edit your own files: `tests/`, `execution/`, `seed-data/mcq-questions.ts`, `seed-data/coding-problems.ts`.
- Do not run student code on the server (`eval`, `new Function`, `vm`, `child_process`).
- Do not send expected outputs to Judge0 or to the student.
- Do not switch to mock mode automatically when Judge0 fails.
- Do not write attempt, run-count, or scoring logic (Rami).
- Do not put a key in Git or in the frontend. Do not write frontend code.
- Check before every PR: `grep -rn "eval(\|new Function\|from 'vm'\|child_process" api/src` → nothing.

6 Presentation Part

Your part in the English presentation: **Run vs. Submit, hidden tests, the execution service, readable errors, mock mode**. If the execution host is unstable on the day, present in mock mode and explain the Judge0 mode with the recorded spike. Prepare your part and one rule from another member's area.